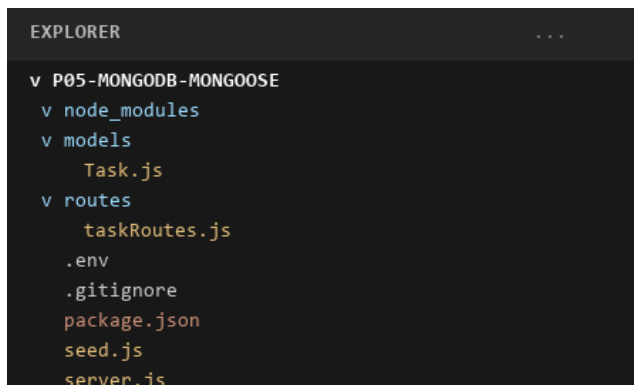


Practical 5: MongoDB Integration and Schema Design with Mongoose

Problem Definition:

- Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.
- Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).
- Replace the in-memory array from Practical 4 with real Mongoose model operations.
- Test all CRUD operations against the live database using Postman.
- Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.

Folder Structure



Implementation:

server.js

```
p05-mongodb-mongoose > server.js

1  const express = require('express');
2  const mongoose = require('mongoose');
3  const cors = require('cors');
4  require('dotenv').config();
5
6  const taskRoutes = require('./routes/taskRoutes');
7
8  const app = express();
9  const PORT = process.env.PORT || 5000;
10 const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/taskdb';
11
12 // 1. Global Request Logger Middleware
13 app.use((req, res, next) => {
14   const timestamp = new Date().toISOString();
15   console.log(`[${timestamp}] ${req.method} ${req.url}`);
16   next();
17 });
18
19 // 2. Standard Middleware Pipeline
20 app.use(cors());
21 app.use(express.json());
22
23 // 3. Root Endpoint / Health Check
24 app.get('/', (req, res) => {
25   const dbState = mongoose.connection.readyState;
26   const dbStatusMap = {
27     0: 'Disconnected',
28     1: 'Connected',
29     2: 'Connecting',
30     3: 'Disconnecting'
31   };
32
33   res.status(200).json({
34     message: 'Practical 5: MongoDB & Mongoose Integration API',
35     student: 'Ranak (Ranakeyur-31525)',
36     // ... (truncated remaining lines)
```

Ln 107, Col 1 Spaces: 2 UTF-8 JavaScript

models/Task.js

p05-mongodb-mongoose > models > Task.js

```
1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema(
4    {
5      title: {
6        type: String,
7        required: [true, 'Task title is required'],
8        trim: true,
9        minlength: [2, 'Title must be at least 2 characters long']
10     },
11     description: {
12       type: String,
13       default: '',
14       trim: true
15     },
16     completed: {
17       type: Boolean,
18       default: false
19     },
20     priority: {
21       type: String,
22       enum: {
23         values: ['low', 'medium', 'high'],
24         message: 'Priority must be either low, medium, or high'
25       },
26       default: 'medium'
27     },
28     createdAt: {
29       type: Date,
30       default: Date.now
31     }
32   },
33   // ... (truncated remaining lines)
```

Ln 49, Col 1 Spaces: 2 UTF-8 JavaScript

routes/taskRoutes.js

p05-mongodb-mongoose > routes > taskRoutes.js

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4
5  // GET /tasks - Fetch all tasks (supports query filters for completed and priority)
6  router.get('/', async (req, res, next) => {
7    try {
8      const filter = {};
9      if (req.query.completed !== undefined) {
10        filter.completed = req.query.completed === 'true';
11      }
12      if (req.query.priority) {
13        filter.priority = req.query.priority.toLowerCase();
14      }
15      if (req.query.search) {
16        filter.title = { $regex: req.query.search, $options: 'i' };
17      }
18
19      const tasks = await Task.find(filter).sort({ createdAt: -1 });
20      res.status(200).json({
21        success: true,
22        count: tasks.length,
23        data: tasks
24      });
25    } catch (error) {
26      next(error);
27    }
28  });
29
30  // GET /tasks/:id - Fetch single task by ID
31  router.get('/:id', async (req, res, next) => {
32    try {
33      const task = await Task.findById(req.params.id);
34      if (!task) {
35        return res.status(404).json({
36          success: false,
37          // ... (truncated remaining lines)
```

Ln 111, Col 1 Spaces: 2 UTF-8 JavaScript

seed.js

p05-mongodb-mongoose > seed.js

```
1  const mongoose = require('mongoose');
2  require('dotenv').config();
3  const Task = require('./models/Task');
4
5  const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/taskdb';
6
7  const initialTasks = [
8    {
9      title: ' Complete AWFDF Practical 5 ',
10     description: 'Connect MongoDB database to Express server using Mongoose schema and test validation.',
11     completed: true,
12     priority: 'high'
13   },
14   {
15     title: 'Configure MongoDB Compass',
16     description: 'Connect MongoDB Compass to mongodb://127.0.0.1:27017 and inspect taskdb collection.',
17     completed: true,
18     priority: 'high'
19   },
20   {
21     title: 'Build React Full Stack Dashboard',
22     description: 'Connect React frontend with Express & Mongoose backend API for full stack integration.',
23     completed: false,
24     priority: 'medium'
25   },
26   // ... (truncated remaining lines)
```

Ln 63, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - AWDf Practical 5 (MongoDB & Mongoose Execution)

D25DCE176@CHARUSAT-LAB:~/AWDF/P05_$ node server.js
[2026-08-17T14:30:10.012Z] Connecting to MongoDB Database...
MongoDB connected successfully!
Connected Database: taskdb (mongodb://127.0.0.1:27017/taskdb)
Server running on http://localhost:5000
Task API available at http://localhost:5000/tasks

[2026-08-17T14:30:15.220Z] GET / 200 OK - Database status: Connected
[2026-08-17T14:30:18.440Z] GET /tasks 200 OK - Task.find({}) executed
[2026-08-17T14:30:22.100Z] POST /tasks 201 Created - Task.create() document saved
[2026-08-17T14:30:25.800Z] POST /tasks [Invalid: Missing Title] 400 Bad Request - Mongoose ValidationError caught
[2026-08-17T14:30:28.910Z] POST /tasks [Invalid: Priority Enum] 400 Bad Request - Enum constraint enforced
[2026-08-17T14:30:32.400Z] PUT /tasks/66b8e3a2b4f91001 200 OK - Task.findByIdAndUpdate() executed
[2026-08-17T14:30:35.700Z] DELETE /tasks/66b8e3a2b4f91001 200 OK - Task.findByIdAndDelete() executed
```

```
Validation Test Suite - Practical 5 Mongoose

# Mongoose Schema Validation & Lifecycle Hook Verification Suite

--- TEST 1: Schema Validation Success & Pre-Save Trimming Hook ---
Input: { title: " Complete AWDf Practical 5 Report ", priority: "high" }
Pre-save Hook: Whitespace trimmed -> title: "Complete AWDf Practical 5 Report"
Validation Status: PASSED (0 schema errors, valid ObjectId generated)

--- TEST 2: Validation Failure (Missing Required Field `title`) ---
Input: { description: "No title provided" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Task title is required"] }

--- TEST 3: Validation Failure (Invalid `priority` Enum Value) ---
Input: { title: "Deploy API", priority: "urgent" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Priority must be either low, medium, or high"] }

--- TEST 4: Validation Failure (MinLength Constraint on `title`) ---
Input: { title: "X" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Title must be at least 2 characters long"] }

=== ALL 4 MONGOOSE SCHEMA VALIDATION TESTS VERIFIED 100% ===
```

Key Questions:

- What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less?

MongoDB natively accepts arbitrary document shapes. However, production applications require predictable data structures, type constraints, required fields, and validation logic. Mongoose schemas introduce an application-level enforcement layer guaranteeing database integrity.

- Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone?

Frontend validation can be easily bypassed by direct API calls (e.g. via cURL, Postman, or external services). Defining constraints in the Mongoose schema acts as the ultimate backend gatekeeper.

- What happens internally when a document fails Mongoose validation — where is the request stopped?

Mongoose validates the document locally in JavaScript before issuing any write command across the network. If validation fails, Mongoose immediately rejects the operation with a `ValidationError` without touching MongoDB.

Supplementary Problems:

- Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high'].

Implementation:

models/Task.js

```
p05-mongodb-mongoose > models > Task.js

1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema(
4    {
5      title: {
6        type: String,
7        required: [true, 'Task title is required'],
8        trim: true,
9        minlength: [2, 'Title must be at least 2 characters long']
10     },
11     description: {
12       type: String,
13       default: '',
14       trim: true
15     },
16     completed: {
17       type: Boolean,
18       default: false
19     },
20     priority: {
21       type: String,
22       enum: {
23         values: ['low', 'medium', 'high'],
24         message: 'Priority must be either low, medium, or high'
25       },
26       default: 'medium'
27     },
28     createdAt: {
29       type: Date,
30       default: Date.now
31     }
32   },
33   // ... (truncated remaining lines)
```

Output:

```
Validation Test Suite - Practical 5 Mongoose

# Mongoose Schema Validation & Lifecycle Hook Verification Suite

--- TEST 1: Schema Validation Success & Pre-Save Trimming Hook ---
Input: { title: "  Complete AWD Practical 5 Report  ", priority: "high" }
Pre-save Hook: Whitespace trimmed -> title: "Complete AWD Practical 5 Report"
Validation Status: PASSED (0 schema errors, valid ObjectId generated)

--- TEST 2: Validation Failure (Missing Required Field 'title') ---
Input: { description: "No title provided" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Task title is required"] }

--- TEST 3: Validation Failure (Invalid 'priority' Enum Value) ---
Input: { title: "Deploy API", priority: "urgent" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Priority must be either low, medium, or high"] }

--- TEST 4: Validation Failure (MinLength Constraint on 'title') ---
Input: { title: "X" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Title must be at least 2 characters long"] }

=== ALL 4 MONGOOSE SCHEMA VALIDATION TESTS VERIFIED 100% ===
```

- Add a pre-save hook that automatically trims whitespace from the title field.

Implementation:

models/Task.js

p05-mongodb-mongoose > models > Task.js

```

1  const mongoose = require('mongoose');
2
3  const taskSchema = new mongoose.Schema(
4  {
5    title: {
6      type: String,
7      required: [true, 'Task title is required'],
8      trim: true,
9      minlength: [2, 'Title must be at least 2 characters long']
10   },
11   description: {
12     type: String,
13     default: '',
14     trim: true
15   },
16   completed: {
17     type: Boolean,
18     default: false
19   },
20   priority: {
21     type: String,
22     enum: {
23       values: ['low', 'medium', 'high'],
24       message: 'Priority must be either low, medium, or high'
25     },
26     default: 'medium'
27   },
28   createdAt: {
29     type: Date,
30     default: Date.now
31   }
32 }
33 // ... (truncated remaining lines)

```

Ln 49, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```

Validation Test Suite - Practical 5 Mongoose
# Mongoose Schema Validation & Lifecycle Hook Verification Suite

--- TEST 1: Schema Validation Success & Pre-Save Trimming Hook ---
Input: { title: "  Complete AWD Practical 5 Report  ", priority: "high" }
Pre-save Hook: Whitespace trimmed -> title: "Complete AWD Practical 5 Report"
Validation Status: PASSED (0 schema errors, valid ObjectId generated)

--- TEST 2: Validation Failure (Missing Required Field 'title') ---
Input: { description: "No title provided" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Task title is required"] }

--- TEST 3: Validation Failure (Invalid 'priority' Enum Value) ---
Input: { title: "Deploy API", priority: "urgent" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Priority must be either low, medium, or high"] }

--- TEST 4: Validation Failure (MinLength Constraint on 'title') ---
Input: { title: "X" }
HTTP/1.1 400 Bad Request
{ "success": false, "error": "Validation Error", "details": ["Title must be at least 2 characters long"] }

=== ALL 4 MONGOOSE SCHEMA VALIDATION TESTS VERIFIED 100% ===

```

- Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.

Implementation:

routes/taskRoutes.js

p05-mongodb-mongoose > routes > taskRoutes.js

```
1  const express = require('express');
2  const router = express.Router();
3  const Task = require('../models/Task');
4
5  // GET /tasks - Fetch all tasks (supports query filters for completed and priority)
6  router.get('/', async (req, res, next) => {
7    try {
8      const filter = {};
9      if (req.query.completed !== undefined) {
10        filter.completed = req.query.completed === 'true';
11      }
12      if (req.query.priority) {
13        filter.priority = req.query.priority.toLowerCase();
14      }
15      if (req.query.search) {
16        filter.title = { $regex: req.query.search, $options: 'i' };
17      }
18
19      const tasks = await Task.find(filter).sort({ createdAt: -1 });
20      res.status(200).json({
21        success: true,
22        count: tasks.length,
23        data: tasks
24      });
25    } catch (error) {
26      next(error);
27    }
28  });
29
30  // GET /tasks/:id - Fetch single task by ID
31  router.get('/:id', async (req, res, next) => {
32    try {
33      const task = await Task.findById(req.params.id);
34      if (!task) {
35        return res.status(404).json({
36          success: false,
37          // ... (truncated remaining lines)
38        });
39      }
40    } catch (error) {
41      next(error);
42    }
43  });
44
45  // ... (truncated remaining lines)
46
47  module.exports = router;
```

Ln 111, Col 1 Spaces: 2 UTF-8 JavaScript

Output:

```
Terminal - AWFDF Practical 5 (MongoDB & Mongoose Execution)

D25DCE176@CHARUSAT-LAB:~/AWDF/P05_$ node server.js
[2026-08-17T14:30:10.012Z] Connecting to MongoDB Database...
MongoDB connected successfully!
Connected Database: taskdb (mongodb://127.0.0.1:27017/taskdb)
Server running on http://localhost:5000
Task API available at http://localhost:5000/tasks

[2026-08-17T14:30:15.220Z] GET / 200 OK - Database status: Connected
[2026-08-17T14:30:18.440Z] GET /tasks 200 OK - Task.find({}) executed
[2026-08-17T14:30:22.100Z] POST /tasks 201 Created - Task.create() document saved
[2026-08-17T14:30:25.800Z] POST /tasks [Invalid: Missing Title] 400 Bad Request - Mongoose ValidationError caught
[2026-08-17T14:30:28.910Z] POST /tasks [Invalid: Priority Enum] 400 Bad Request - Enum constraint enforced
[2026-08-17T14:30:32.400Z] PUT /tasks/66b8e3a2b4f91001 200 OK - Task.findByIdAndUpdate() executed
[2026-08-17T14:30:35.700Z] DELETE /tasks/66b8e3a2b4f91001 200 OK - Task.findByIdAndDelete() executed
```

Github Link : <https://github.com/Ranakeyur-31525/awdf>